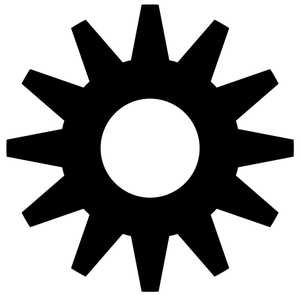
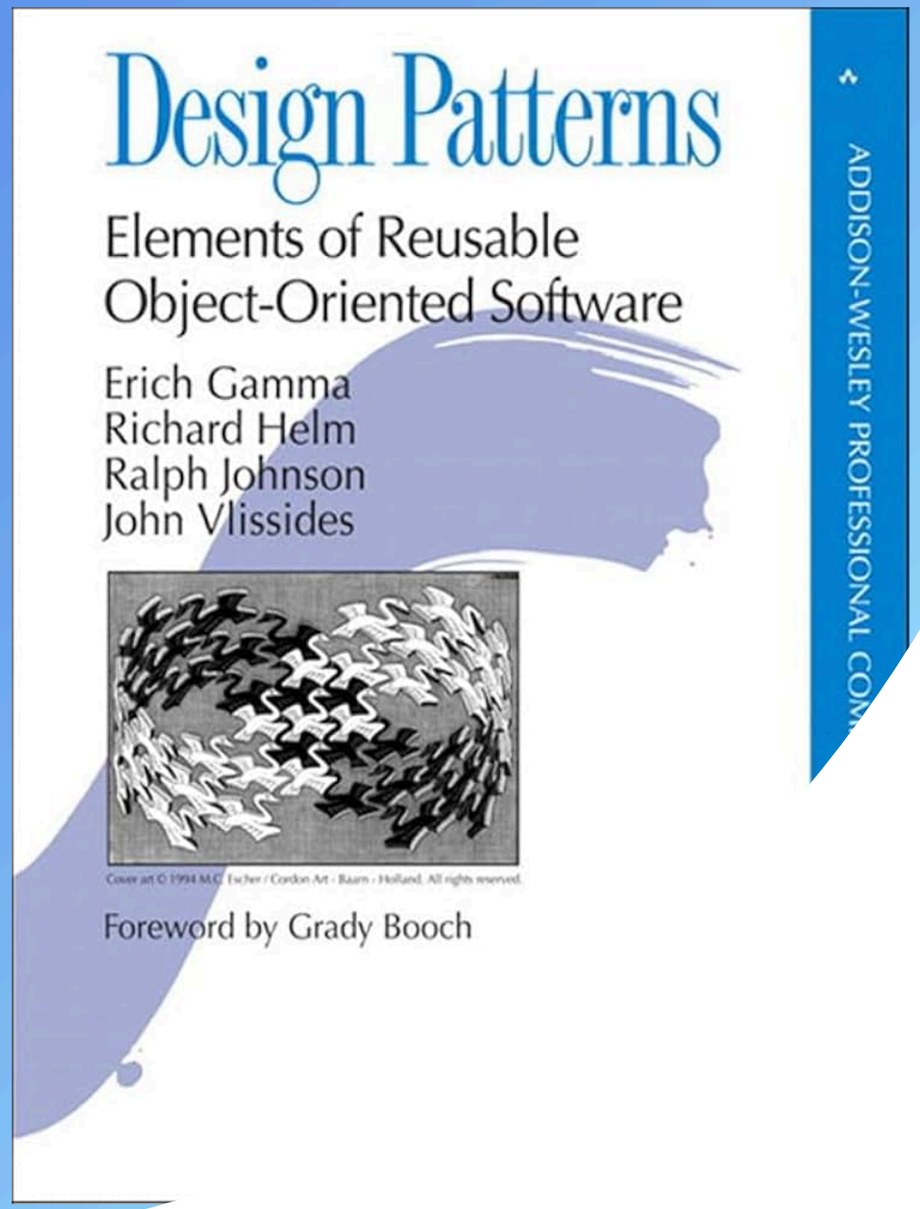
Le design pattern command



Anatole Laspoujas
Ivan Lohvinov
Luc Magnaudeix

Sommaire

1. Introduction sur les patrons de conception
2. Exemple d’un besoin
3. Proposition de conception pour répondre au besoin
4. Problèmes de cette conception
5. Appliquer le patron de conception commande sur cet exemple
6. Explication de ce patron
7. Détail des rôles des classes par rapport à l’exemple
8. Lien avec SOLID
9. Limites du patron
10. Lien avec les patrons Stratégie et Visiteur
11. Deuxième exemple
12. Live coding
13. Conclusion + Bibliographie + IAGraphie



Design patterns
Patrons de conception

Solution à des problèmes de problèmes récurrents dans la conception de d'application

Diviser en **3** familles



Behavioral
Comportemental



Structural
Structurel



Creational
Créateur

1



2



3



4



5



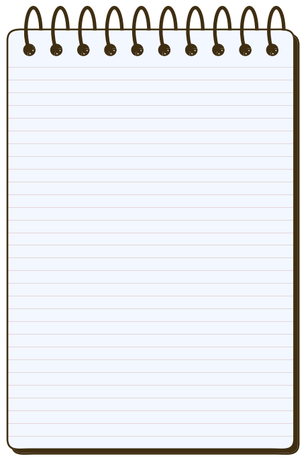
6



1



2



3



4



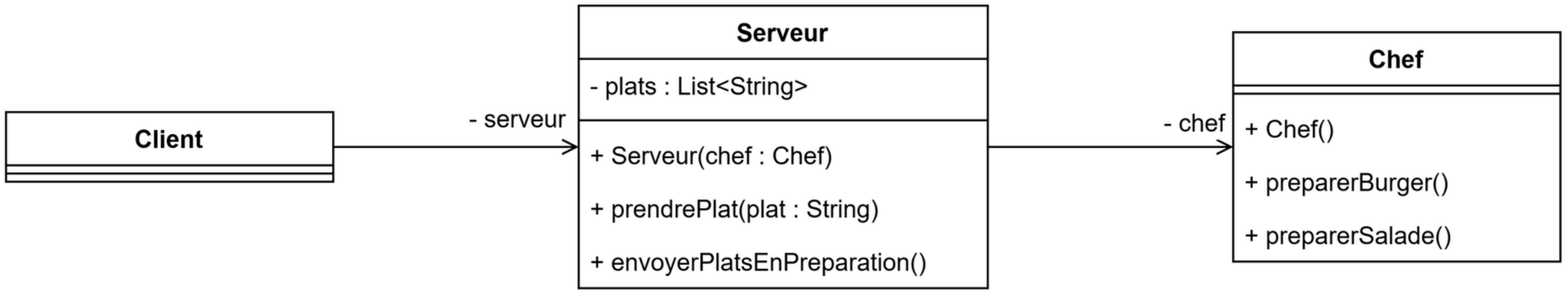
5



6



Première modélisation pour répondre au besoin



Première modélisation pour répondre au besoin

```java
public class Serveur {
    private final List<String> plats;
    private final Chef chef;

    public Serveur(Chef chef) {
        this.plats = new ArrayList<>();
        this.chef = chef;
    }

    public void prendreCommande(String plat) {
        plats.add(plat);
    }

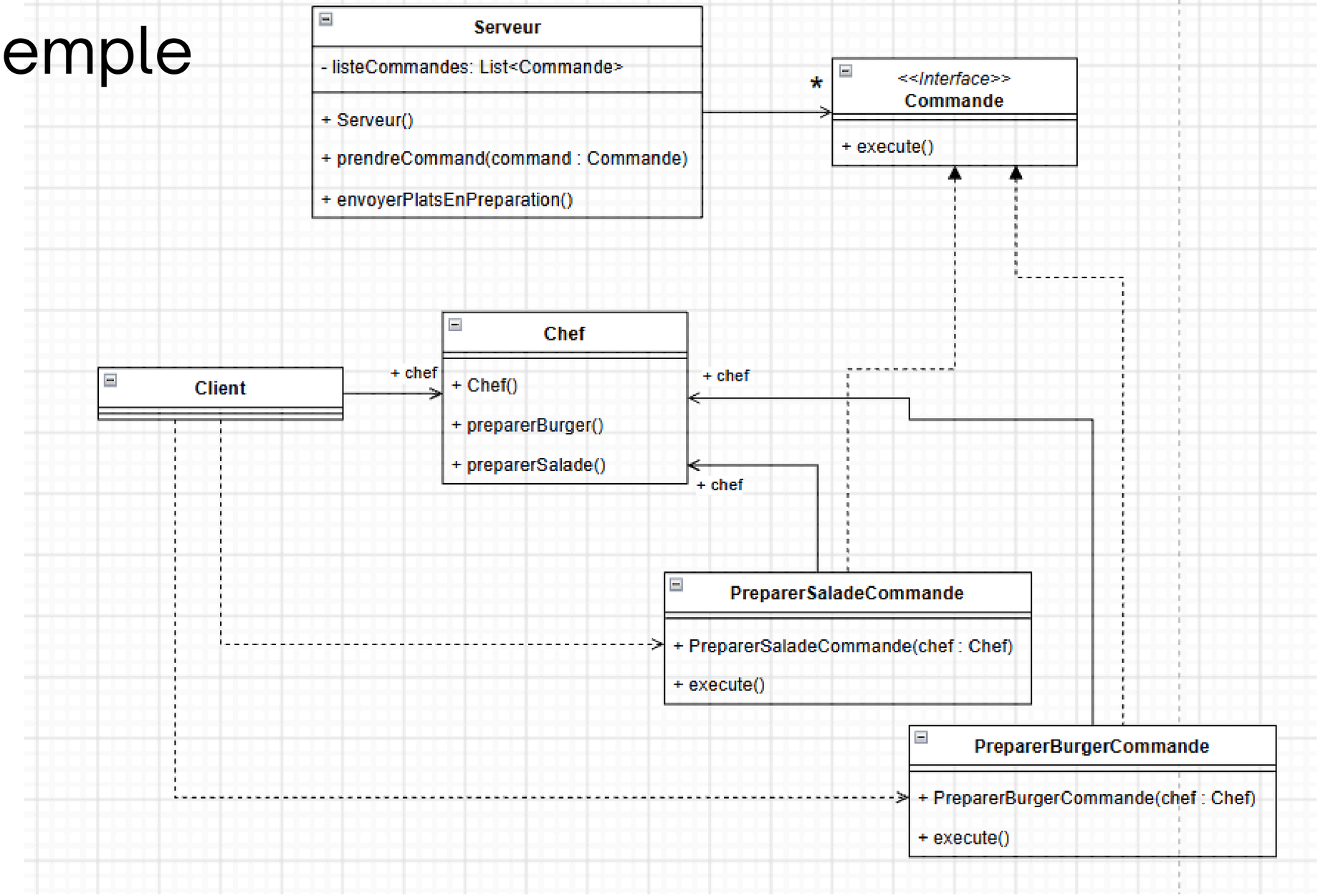
    public void envoyerPlatsEnPreparation() {
        for (String plat : plats) {
            if (plat.equals("Burger")) {
                chef.preparerBurger();
            } else if (plat.equals("Salade")) {
                chef.preparerSalade();
            }
        }
    }
}
```

```java
public class Chef {
    public Chef() {}

    public void preparerBurger() {
        System.out.println("Un bon burger !");
    }

    public void preparerSalade() {
        System.out.println("Une salade bien verte !");
    }
}
```

Appliquer le patron de conception commande sur l'exemple

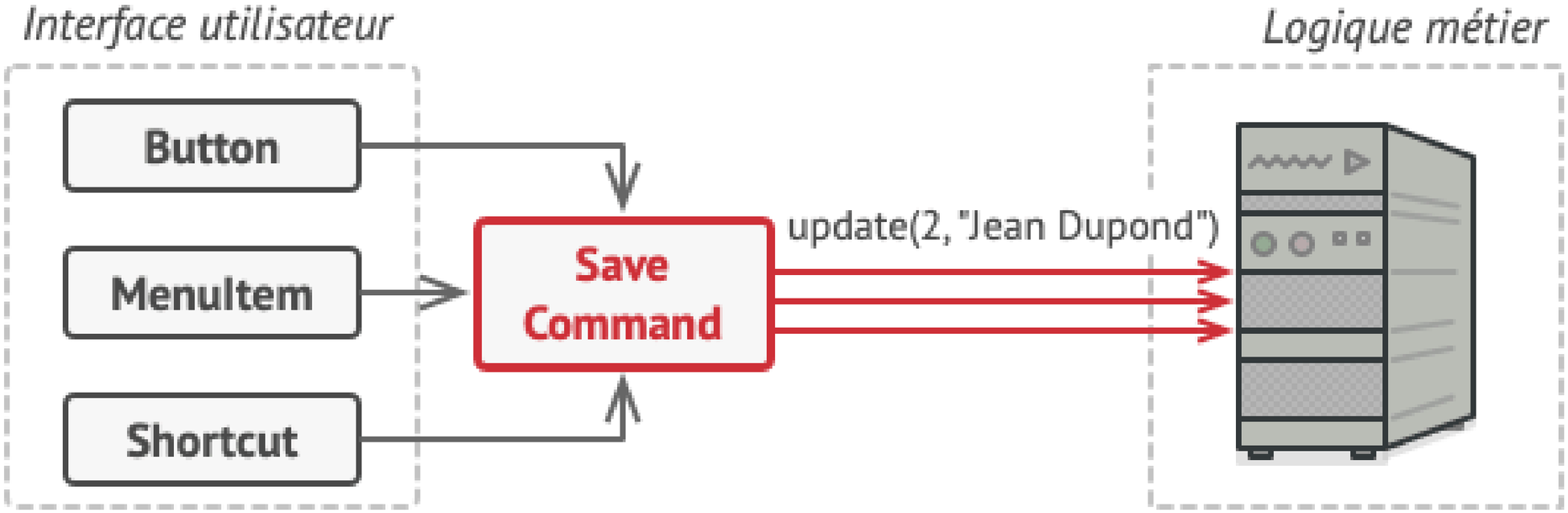


Quand utiliser ce pattern ?



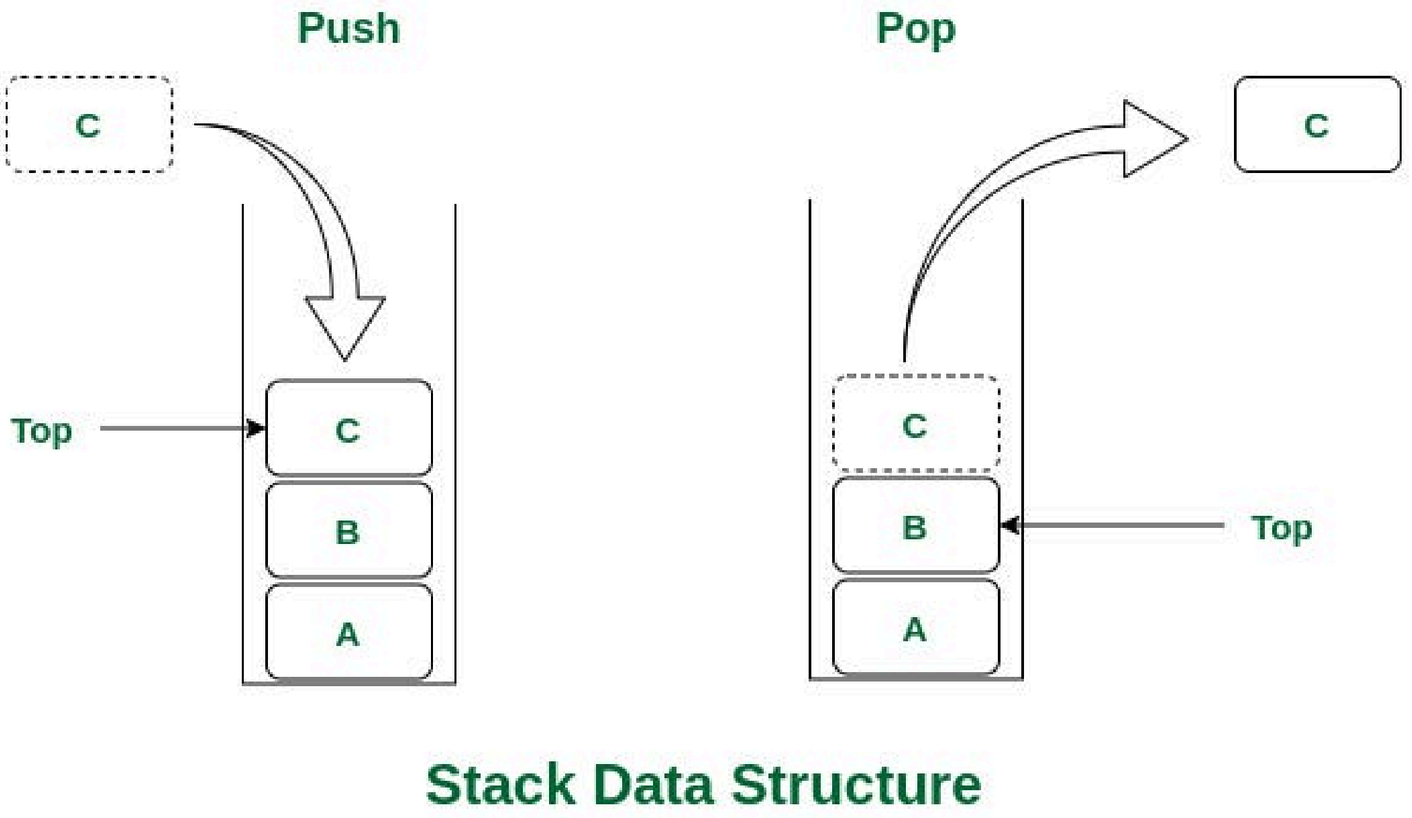
https://waytolearnx.com/2020/03/difference-entre-couplage-faible-et-couplage-fort-en-java.html

Quand utiliser ce pattern ?



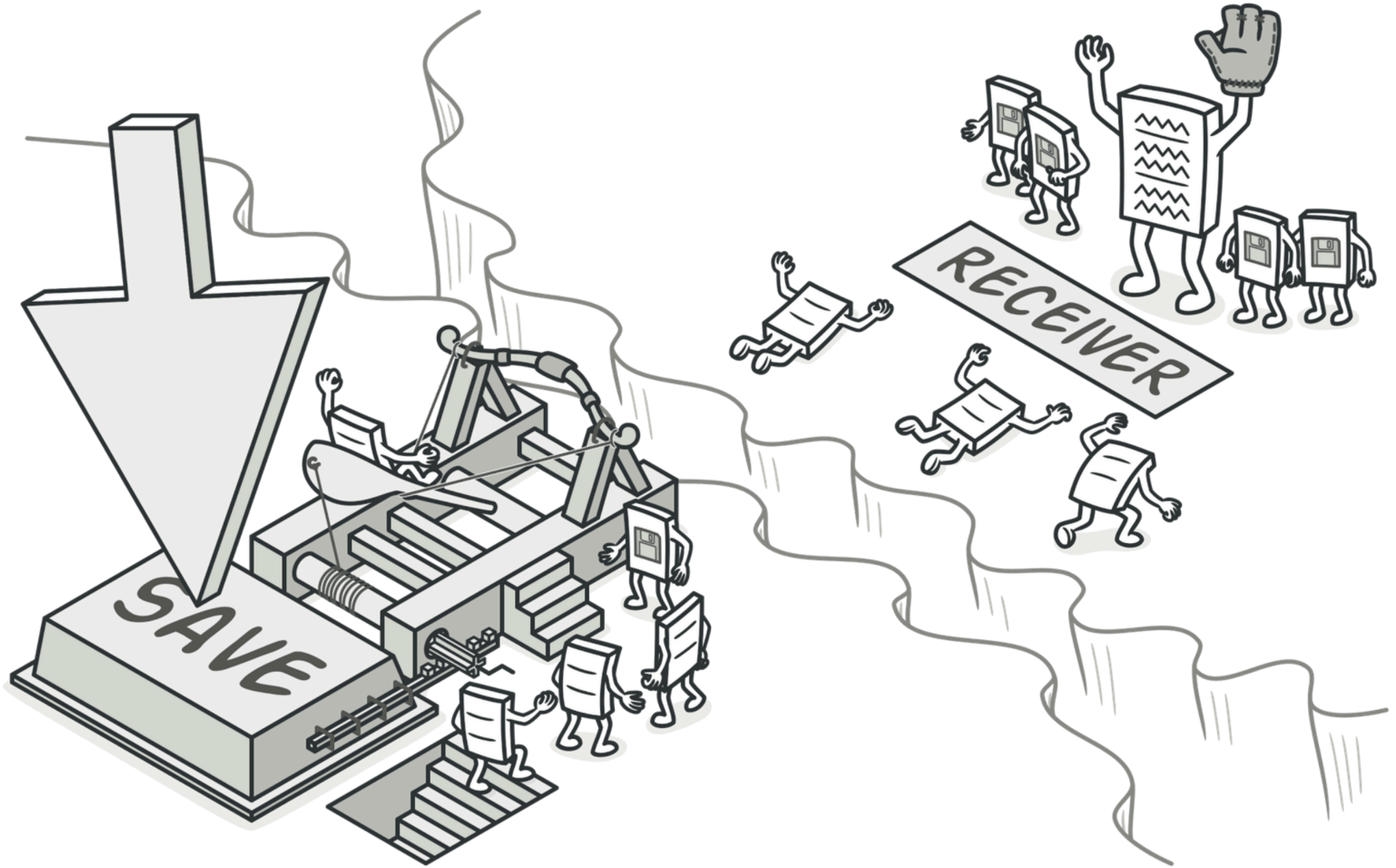
https://refactoring.guru/fr/design-patterns/command

Quand utiliser ce pattern ?



Stack Data Structure

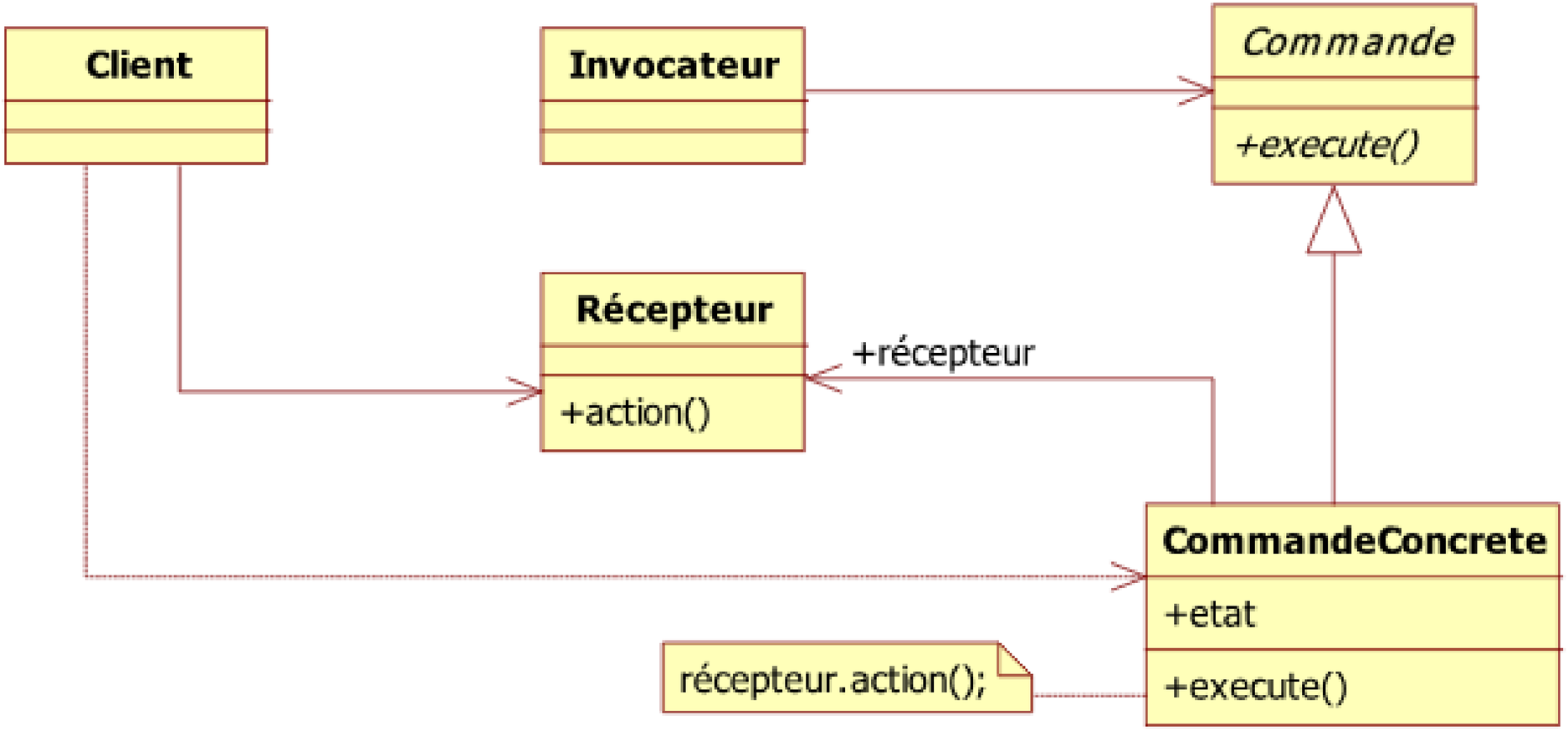
Quelle est l’intention de ce pattern ?



https://refactoring.guru/fr/design-patterns/command

Commande

Diagramme de classes :

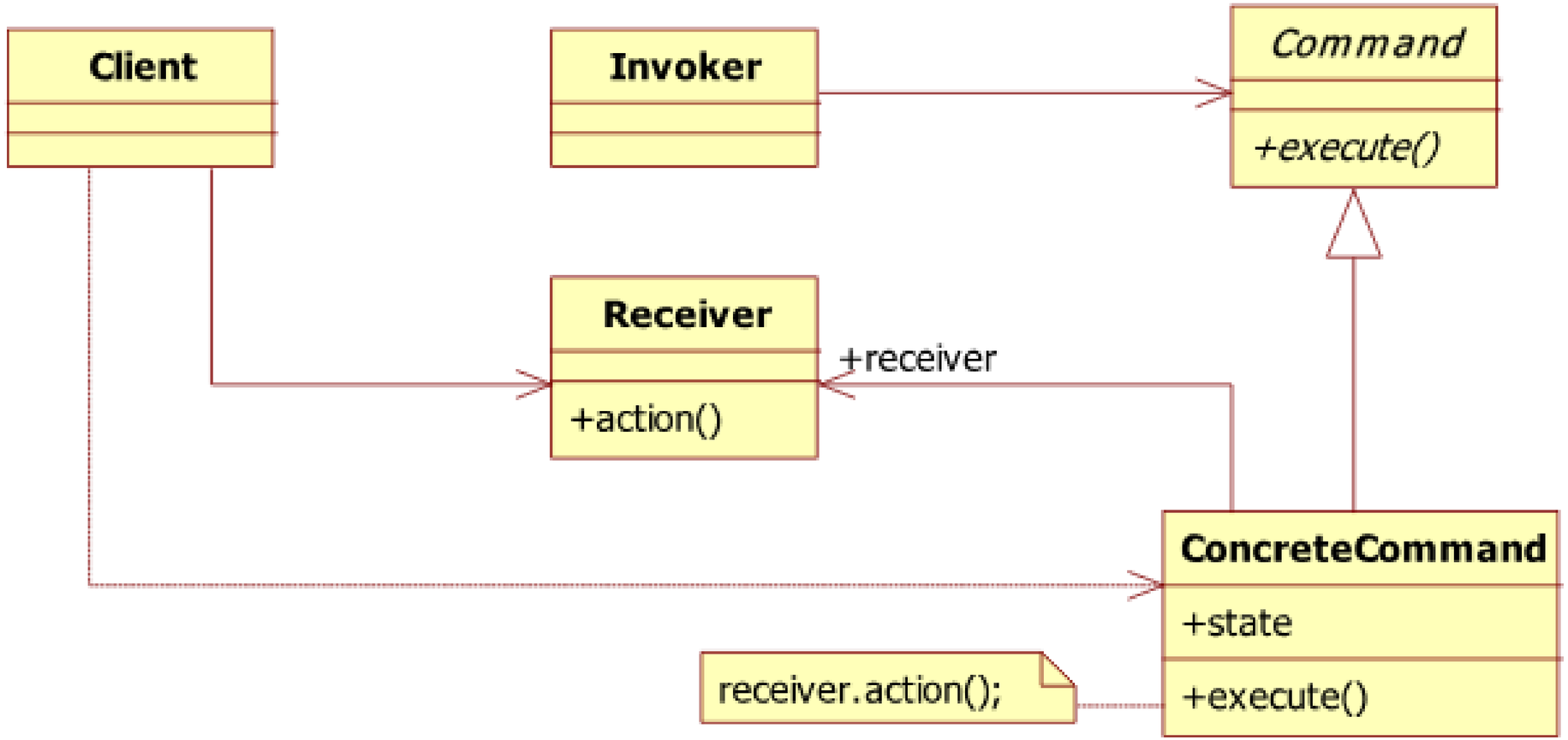


http://www.goprod.bouhours.net/



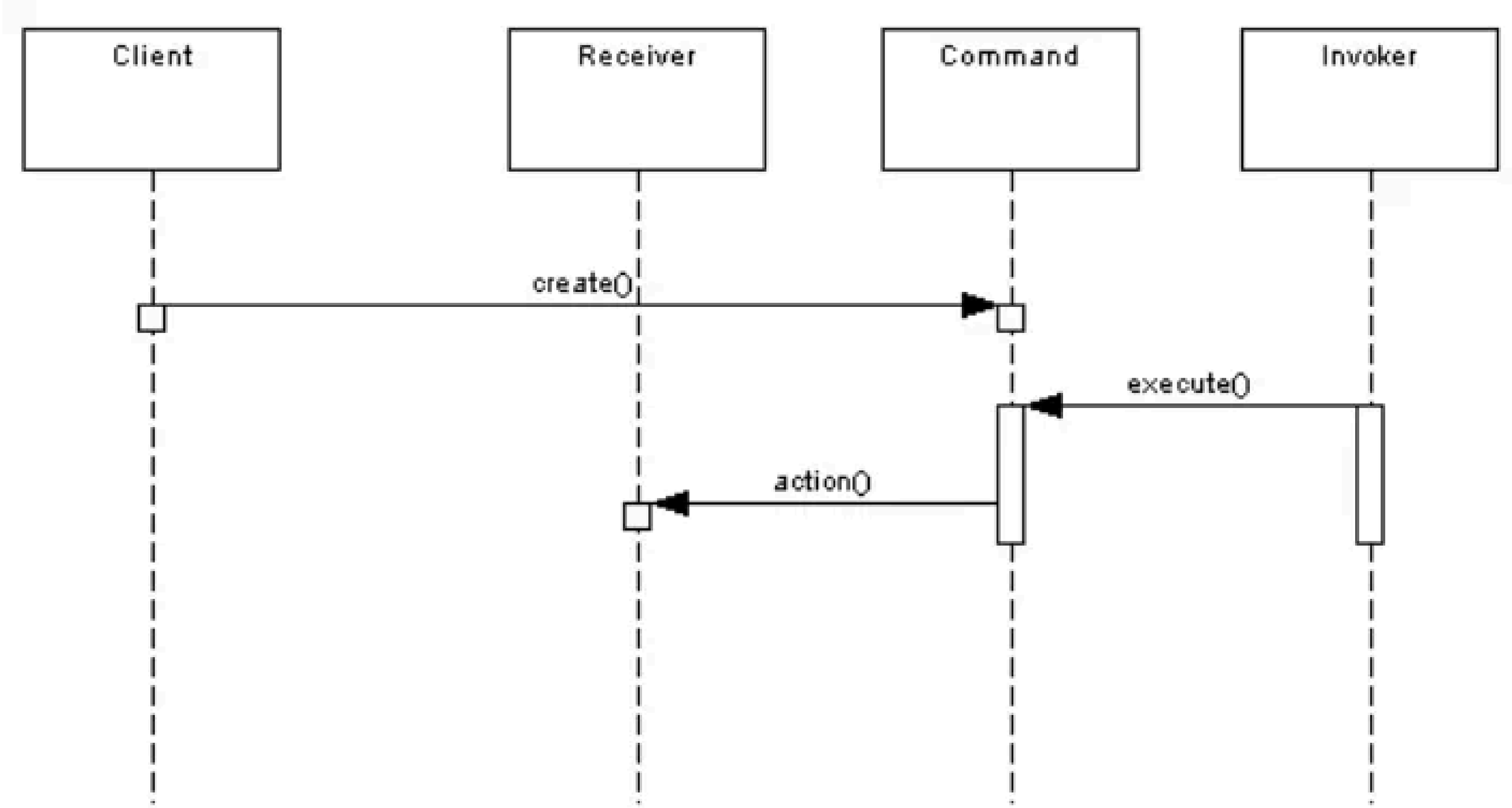
Command

Static diagram :



http://www.goprod.bouhours.net/

Diagramme de séquence



https://medium.com/elp-2018/command-design-pattern-quest-ce-78bc93fb0942

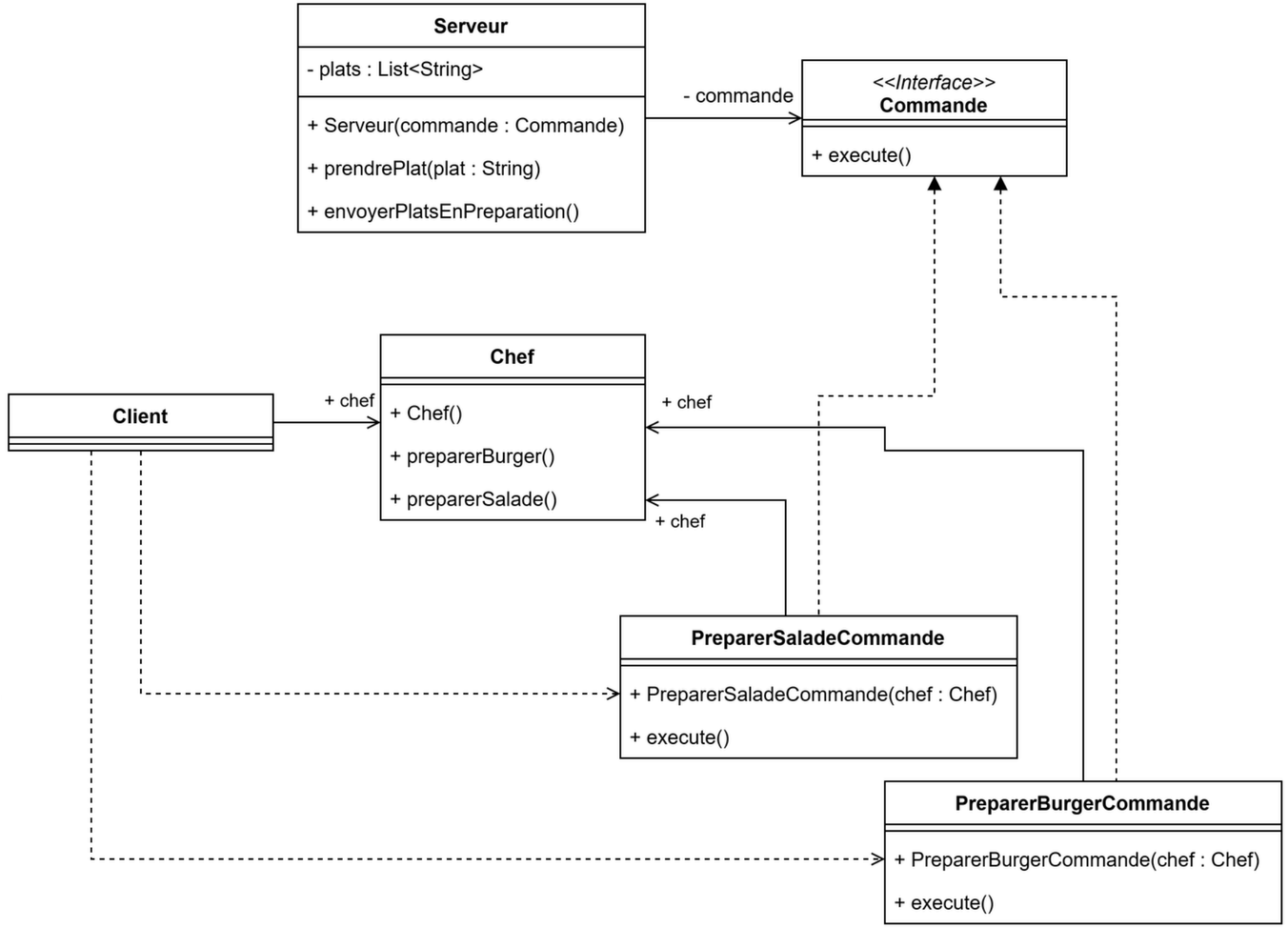
Les noms des classes par rapport à l’exemple du



Nom du design pattern	Nom dans notre exemple
Client	Client
Receiver	Chef
Command	Command
Invoker	Serveur
Concrete Command 1	PreparerBurgerCommande
Concrete Command 2	PreparerSaladeCommande

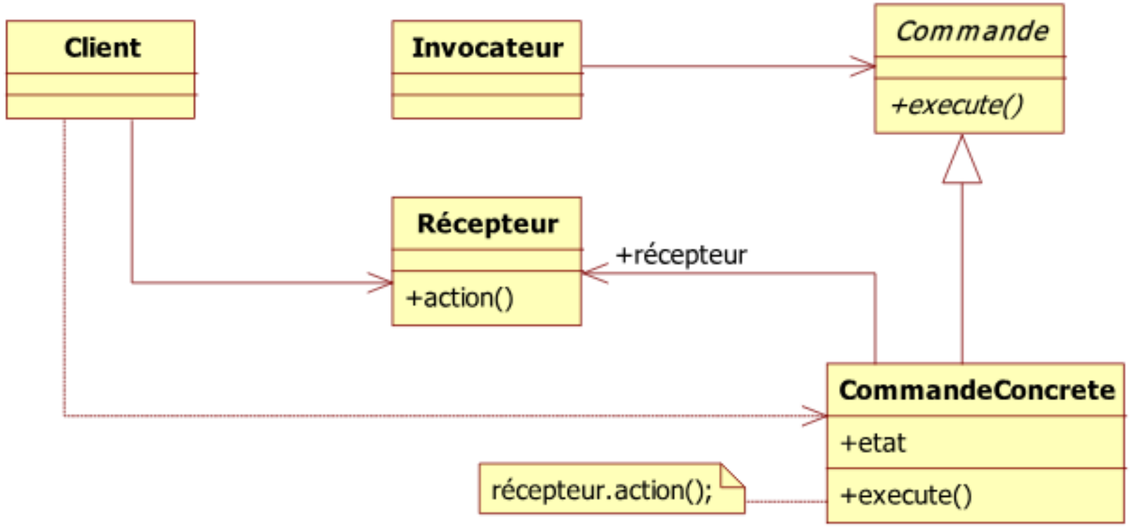
D’où :





Commande

Diagramme de classes :



SOLID dans le pattern Command

- **SRP** - On peut découpler les classes qui appellent des traitements, de celles qui les exécutent, donc chaque classe aura 1 responsabilité
- **OCP –** On peut ajouter de nouvelles commandes dans le programme sans changer le code client existant. Le code est fermé aux modifications mais ouvert à l'extension.
- **LSP** - Toutes les commandes concrètes implémentent la même interface commune Commande. N'importe quelle commande concrète peut être substituée à une autre et injectée dans l'Invocateur sans altérer ni casser le comportement du programme.
- **ISP –** L’interface commande est minimaliste, donc les classes clientes ne dépendent pas de méthodes non utilisées
- **DIP –** Invoker ne dépend pas directement des classes concrètes receiver, mais d'une abstraction (Commande). Les modules de haut et bas niveau dépendent tous deux de cette interface.

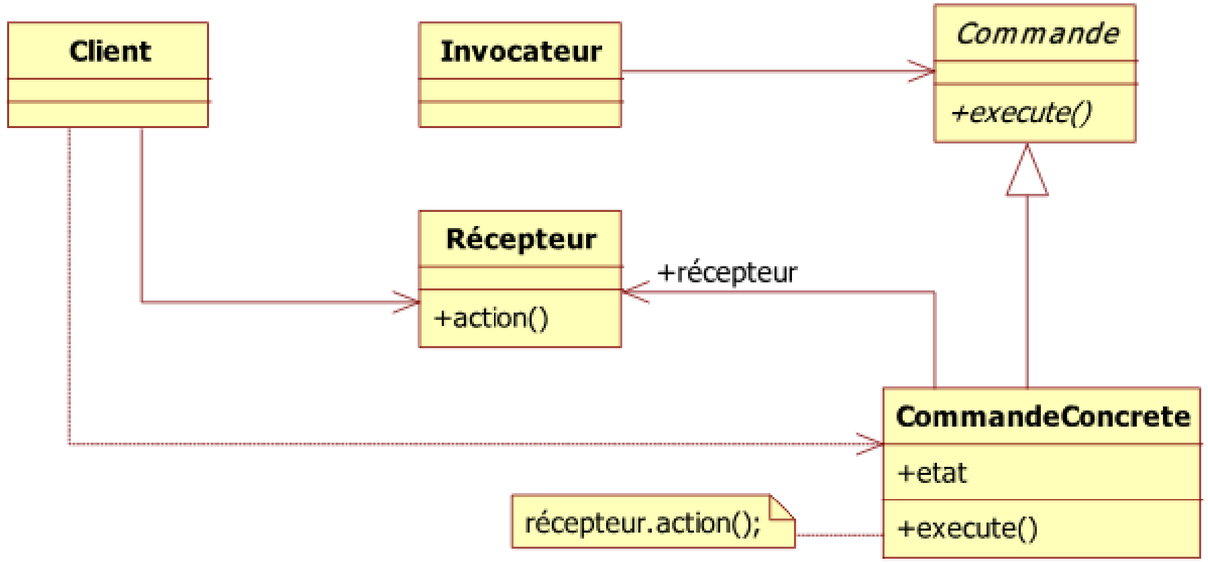
https://fr.linkedin.com/pulse/mastering-solid-principles-shubham-kashyap-5uh6c?tl=fr

Lien entre Commande et Stratégie

Quoi

Commande

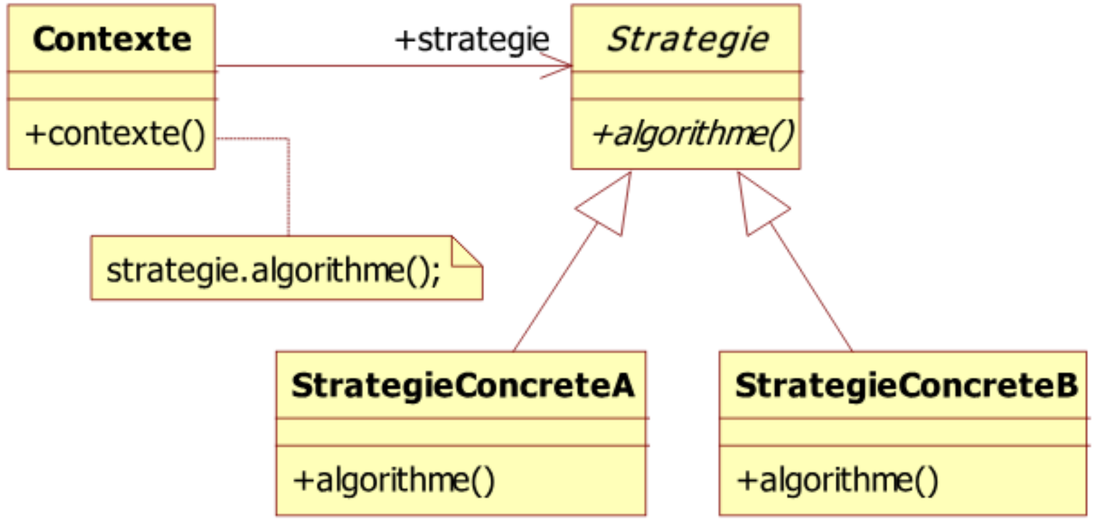
Diagramme de classes :



Comment

Stratégie

Diagramme de classe :



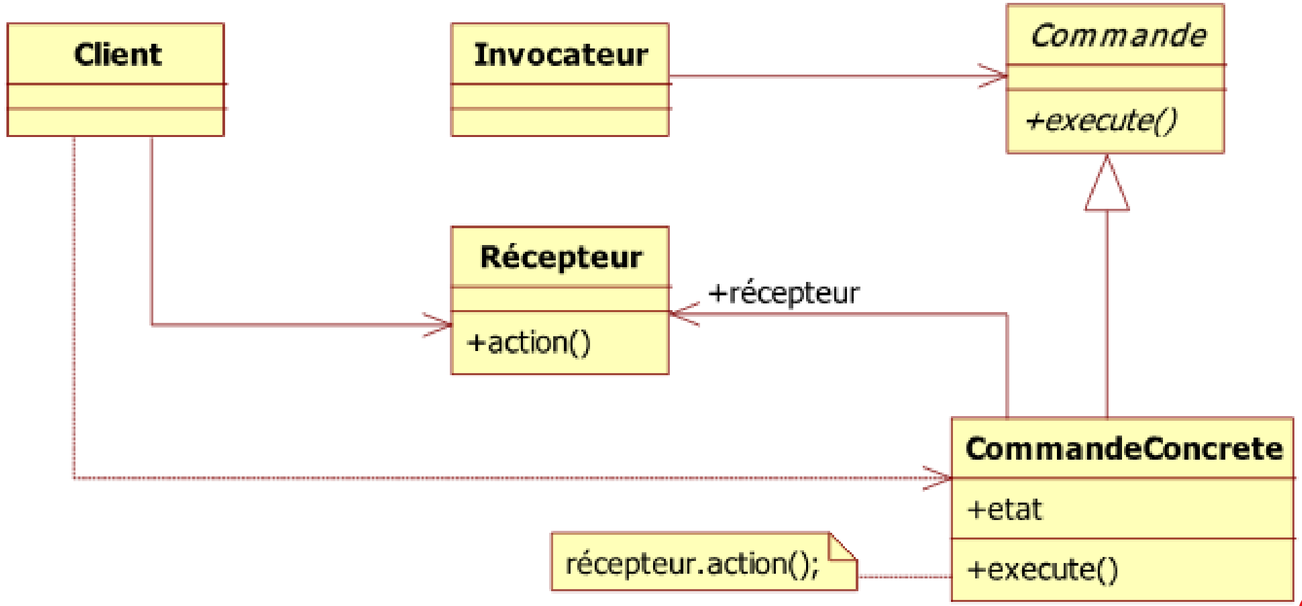
Utilisation de la composition pour encapsuler un comportement

Lien entre Commande et Visiteur

l'action et la dimension temporelle

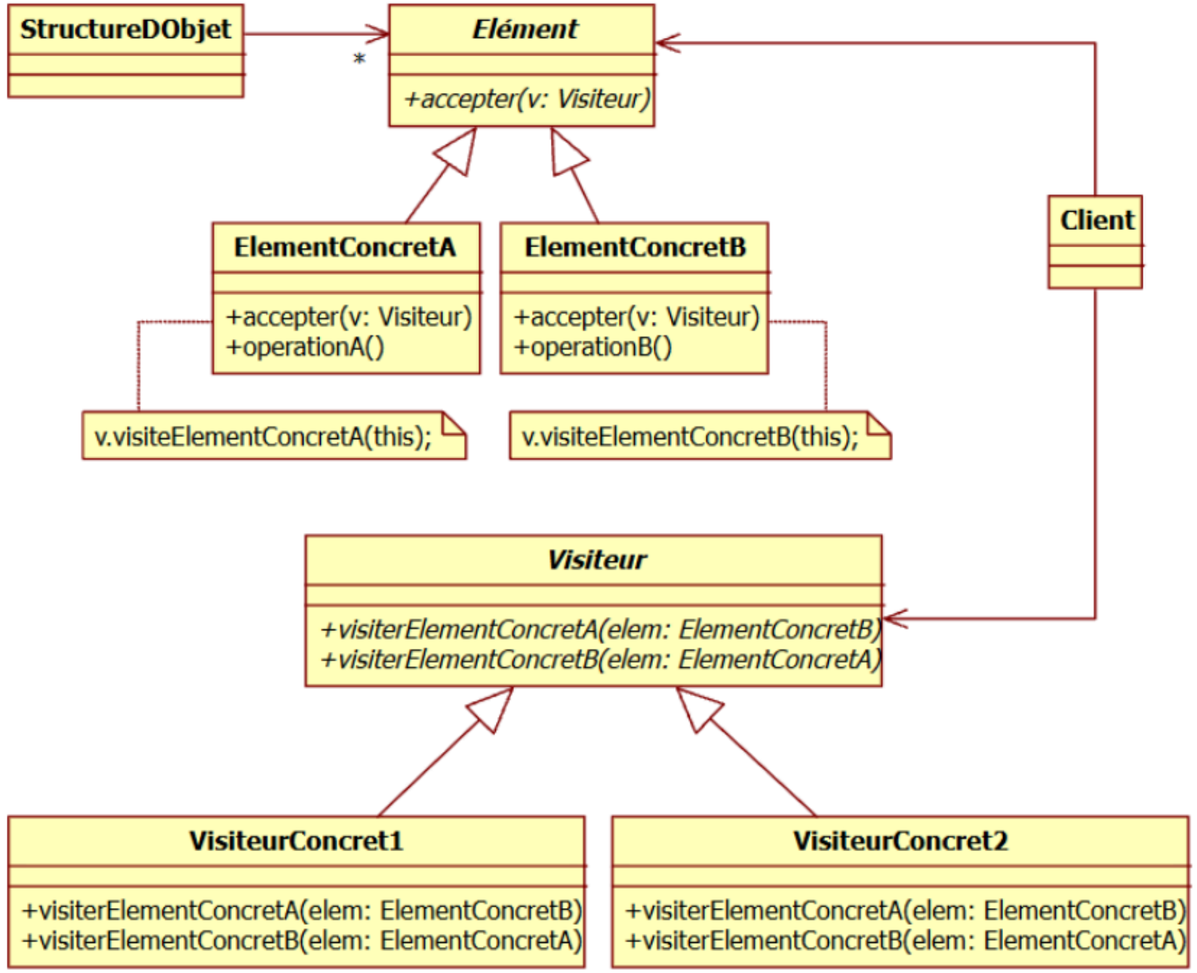
Commande

Diagramme de classes :



Visiteur

Diagramme de classes :



Opérations sur des objets de types différents

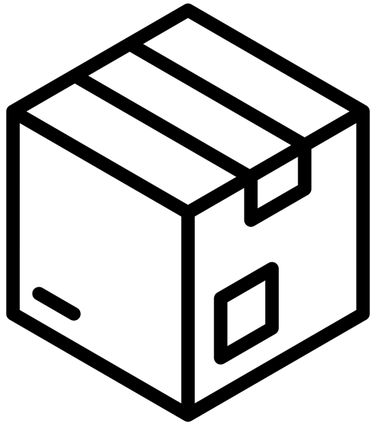
Le design pattern Command dans un jeu vidéo



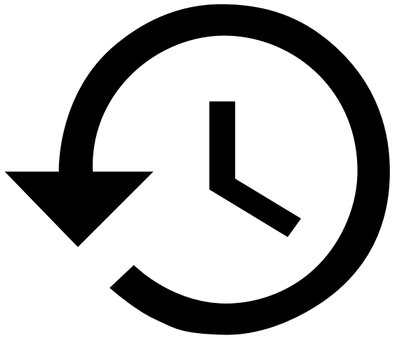
Pourquoi le pattern command?



Historisation



Sauvegarde



Restauration

Roles selon le pattern Commande

Client - Joueur

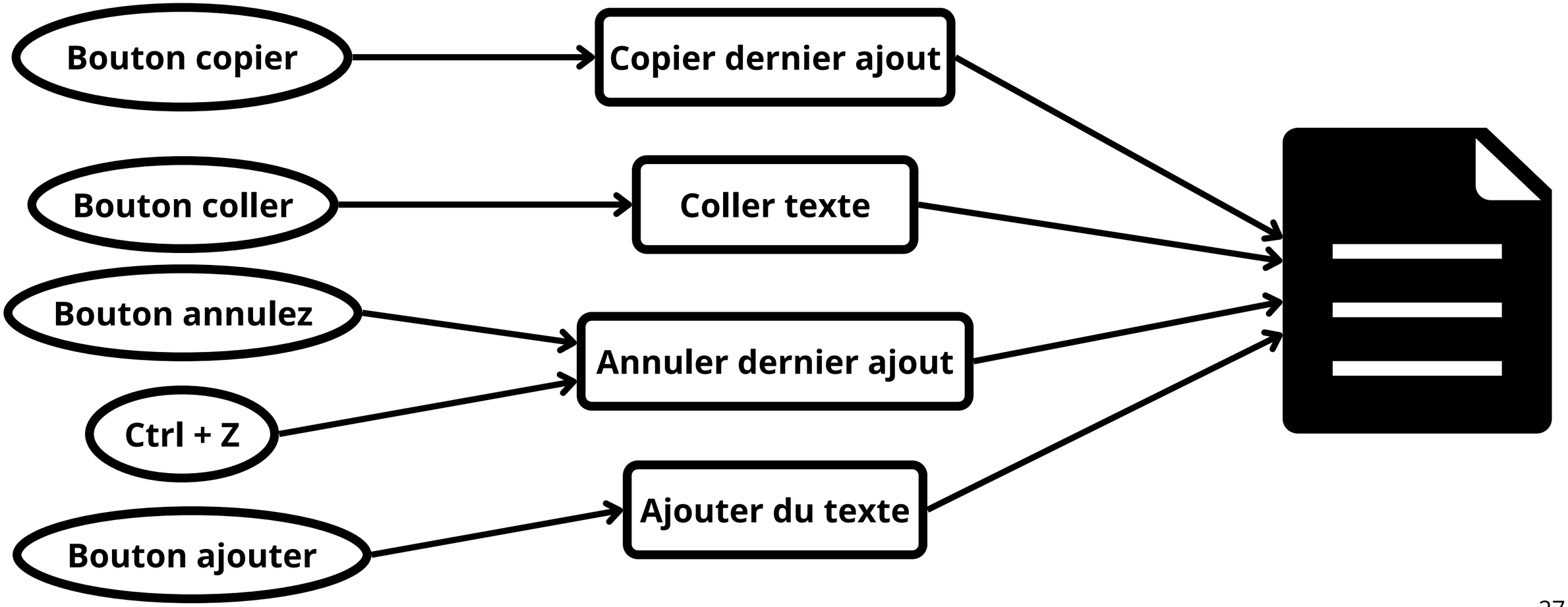
Invocateur - Gestionnaire de commandes

Commande - interface générale

Concrète commande - commande specifique(//copy, //replace)

Récepteur - moteur du monde qui fait tous modifications

Exemple du document



Exemple du document

```java
public class Document {
    public static String pressePapier = "";

    private List<String> contenu;

    public Document() {
        this.contenu = new ArrayList<>();
    }

    public void ajouterDuTexte(String texte) {
        contenu.add(texte);
    }

    public void annulerDernierAjout() {
        if (!contenu.isEmpty()) {
            contenu.removeLast();
        }
    }

                    ...

                    ...

    public void copierDernierAjout() {
        pressePapier = contenu.getLast();
    }

    public void collerDepuisPressePapier() {
        ajouterDuTexte(pressePapier);
    }

    public void afficherDocument() {
        String document = "";

        for (String morceau : contenu) {
            document = document + morceau + " ";
        }

        System.out.println(document);
    }
}
```

Exemple du document

```java
public class Bouton {
    private Commande commande;

    public Bouton(Commande commande) { this.commande = commande; }

    public void appuyer() { commande.execute(); }
}

public class Raccourci {
    private Commande commande;

    public Raccourci(Commande commande) { this.commande = commande; }

    public void utiliser() {
        commande.execute();
    }
}
```

```java
public interface Commande {
    public void execute();
}
```

Exemple du document

```java
public class CopierDernierAjoutCommande implements Commande {
    private Document document;

    public CopierDernierAjoutCommande(Document document) { this.document = document; }

    @Override
    public void execute() {
        document.copierDernierAjout();
    }
}
```

```java
 public class CollerCommande implements Commande {
     private Document document;

     public CollerCommande(Document document) { this.document = document; }

     @Override
     public void execute() {
         document.collerDepuisPressePapier();
     }
 }
```

Exemple du document

```java
public class AnnulerDernierAjoutCommande implements Commande {
    private Document document;

    public AnnulerDernierAjoutCommande(Document document) { this.document = document; }

    @Override
    public void execute() {
        document.annulerDernierAjout();
    }
}
```

```java
public class AjouterTexteCommande implements Commande {
    private Document document;
    private String texte;

    public AjouterTexteCommande(Document document, String texte) {
        this.document = document;
        this.texte = texte;
    }

    @Override
    public void execute() {
        document.ajouterDuTexte(texte);
    }
}
```

Exemple du document

```
public class Main {
    public static void main(String[] args) {
        Document document = new Document();

        Bouton btnCopier = new Bouton(
                new CopierDernierAjoutCommande(document)
        );
        Bouton btnColler = new Bouton(
                new CollerCommande(document)
        );
        Bouton btnAnnuler = new Bouton(
                new AnnulerDernierAjoutCommande(document)
        );
        Bouton btnAjouterHello = new Bouton(
                new AjouterTexteCommande(document, texte: "Hello")
        );
        Bouton btnAjouterWorl = new Bouton(
                new AjouterTexteCommande(document, texte: "World")
        );

        Raccourci rccAnnuler = new Raccourci(
                new AnnulerDernierAjoutCommande(document)
        );
                                ...
```

Exemple du document

```
...
        System.out.println("Test ajout :");
        btnAjouterHello.appuyer();
        document.afficherDocument();
        btnAjouterWorl.appuyer();
        document.afficherDocument();
        System.out.println();

        System.out.println("Test copier et coller :");
        btnCopier.appuyer();
        btnColler.appuyer();
        document.afficherDocument();
        btnColler.appuyer();
        document.afficherDocument();
        System.out.println();

        System.out.println("Test annuler :");
        btnAnnuler.appuyer();
        document.afficherDocument();
        rccAnnuler.utiliser();
        document.afficherDocument();
    }
}
```

```
Test ajout :
Hello
Hello World

Test copier et coller :
Hello World World
Hello World World World

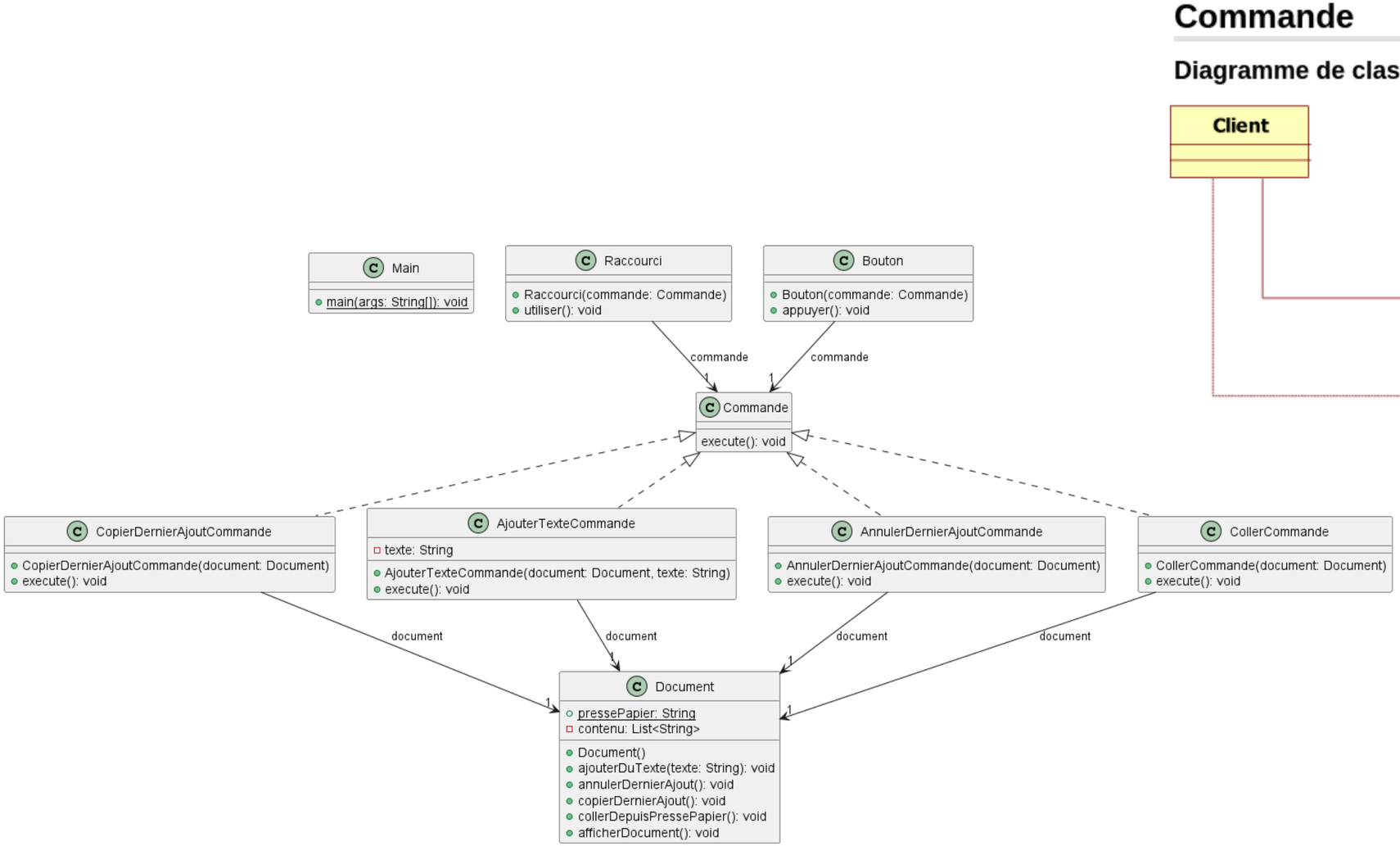
Test annuler :
Hello World World
Hello World

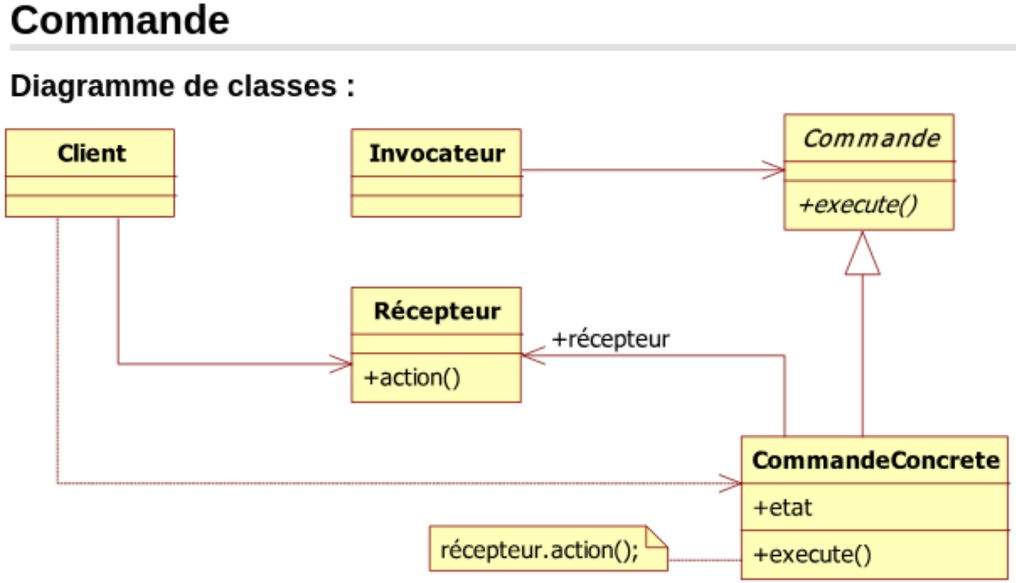
Process finished with exit code 0
```

Exemple du document

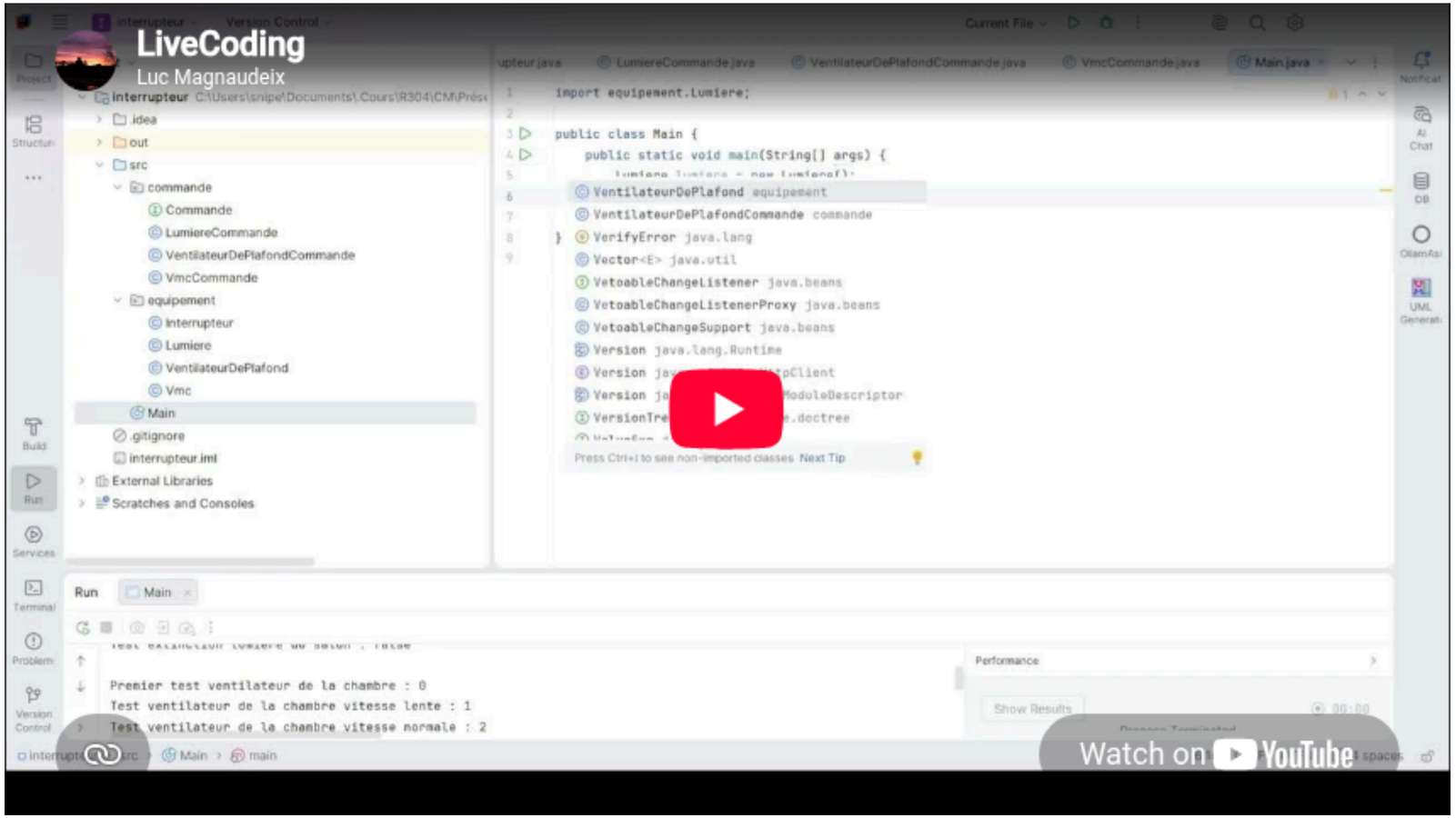


Exemple du document





Live coding


LiveCoding
Luc Magnaudeix
interrupteur
.idea
out
src
commande
Commande
LumiereCommande
VentilateurDePlafondCommande
VmcCommande
equipement
Interrupteur
Lumiere
VentilateurDePlafond
Vmc
Main
.gitignore
interrupteur.iml
External Libraries
Scratches and Consoles
import equipement.Lumiere;
public class Main {
public static void main(String[] args) {
VentilateurDePlafond equipement
VentilateurDePlafondCommande commande
VerifyError java.lang
Vector<E> java.util
VetoableChangeListener java.beans
VetoableChangeListenerProxy java.beans
VetoableChangeSupport java.beans
Version java.lang.Runtime
Press Ctrl+I to see non-imported classes Next Tip
Run
Main
Premier test ventilateur de la chambre : 0
Test ventilateur de la chambre vitesse lente : 1
Test ventilateur de la chambre vitesse normale : 2
Performance
Show Results
Watch on YouTube

QCM



Bibliographie

http://www.goprod.bouhours.net/?lang=fr&
https://refactoring.guru/fr/design-patterns/command
https://refactoring.guru/design-patterns/strategy
https://refactoring.guru/design-patterns/visitor
https://youtu.be/5M-v8LbLI8c?si=HZddwhmifPj4SdR3
https://waytolearnx.com/2020/03/difference-entre-couplage-faible-et-couplage-fort-en-java.html
https://reactiveprogramming.io/blog/en/design-patterns/command
https://www.youtube.com/watch?v=3OKobBWfJSQ
https://www.youtube.com/watch?v=UfGD60BYzPM



IAGraphie

Outil utilisé	Nature de l’apport	Prompt de synthèse
Gemini 3 (auto)	• Clarification de l'intention du pattern et des cas d'usage (explicitations et analogies). • Validation du diagramme de séquence UML (alignement avec les standards GoF). • Relecture critique du support (détection des imprécisions, mise en conformité SOLID). • Relecture, correction des coquilles et mise en forme du QCM préparé par l'équipe.	Relis mon exposé sur le pattern Command : relève les trucs faux ou imprécis, valide mes diagrammes UML et relis le QCM."
Gemma4	• Explication de certains points du pattern	Explique moi à quoi sert *tel point* du pattern.
Gemma4	• Vérification de code pour s’assurer qu’il n’y ai pas de coquille notamment dans le respect du pattern.	Ce code respect-il le pattern command ?
Gemini 3.8	• Corriger des erreurs lexiques et grammatiques	Corriger les fautes d'orthographe et de grammaire!